1 Euler Tour

```cpp
int timer = 0;
// Queries on subtree range [in,out)
function<void(int,int)> euler_tour = [&](int u, int
    p) -> void {
  in[u] = timer++;
  for(int v : adj[u])
  if( v != p )
    euler_tour(v,u);
  out[u] = timer;
};
euler_tour(1,0);
//===== Queries with Fenwick Tree
BIT<ll> bit(V);
for(int u = 1; u <= V; u++) bit.update(in[u]+1,v[u])
    ;
// Update 'node' to 'val' (set operation)
bit.update(in[node]+1,val-v[node]);
v[node] = val;
// Query (ET is 0-idx, but BIT 1-idx)
cout << bit.query(in[node]+1,out[node]) << endl;
//===== Queries with Segment Tree
vll flat(V+1);
for(int i = 1; i <= V; i++)
  flat[ in[i] ] = v[i];
Segment_tree st(V);
st.build(flat,1,0,V-1);
st.update(1,in[node],0,V-1,val);
st.query(1,in[node],out[node]-1,0,V-1);
```

Listing 1: Euler Tour implementation, runs in $O(|V| + |E|)$. Works with an 1-indexed logic, but the time mapping is 0-indexed. Brings necessary logic to work with BIT and ST templates.

2 Coordinate Compression

```cpp
//Previously read vector<int> a(n)
vector<int> coords = a;
sort(all(coords));
coords.erase(unique(all(coords)), coords.end()); //
    erases duplicates
//Reassings every value in vector "a" to the new
    corresponding value
for(int &x : a) x = lower_bound(all(coords), x) -
    coords.begin();
```

Listing 2: Coordinate Compression implementation for a vector of integers. Replaces elements in the original vector to newly assigned minimal numbers. Runs in $O(nlog(n))$.

3 Sqrt Decomposition

3.1 Mo's Algorithm

```cpp
struct Query{
    int l, r, idx;
```

```
};
//vector queries ya leido y 0-indexado anteriormente
int block_size = (int)sqrt(n);
auto mo_cmp = [&](Query a, Query b) { //custom cmptor
    int block_a = a.l / block_size;
    int block_b = b.l / block_size;
    if(block_a != block_b) return block_a < block_b;
    if(block_a & 1) return a.r < b.r; //zig-zag sorting
    return a.r > b.r;
};
sort(all(queries), mo_cmp);
```

Listing 3: Offline query ordering for Mo's Algorithm. Sorts the queries by block of the left endpoint, alternating the right endpoint order per block (zig-zag) to reduce moves. The sort itself runs in $O(q\,log(q))$; the whole algorithm runs in $O((n+q)\sqrt{n}\cdot F)$, where F is the cost of an update (usually $O(1)$ or $O(log(n))$).

4 Input Parsing

```
#include <bits/stdc++.h>
using namespace std;

vector<string> splitLine() {
    string line;
    getline(cin, line);
    vector<string> tokens;
    stringstream ss(line);
    string token;
    while (ss >> token) tokens.push_back(token);
    return tokens;
}

/*
Usage example:
int main() {
    fastIO();
    int n; cin >> n;
    cin.ignore(); // ignore newline after reading n

    for (int i = 0; i < n; i++) {
        vector<string> line = splitLine();
        // process line...
    }
}
*/
```

Listing 4: Parses input line by line. Reads a full line and splits it by spaces into a vector of strings. Complexity: $O(L)$ time and memory, where L is the length of the line.

5 Regex

```
#include <bits/stdc++.h>
using namespace std;
```

```

// First occurrence of pattern in s, or "" if there is none.
string findFirst(string s, string pattern) {
    smatch m;
    if (regex_search(s, m, regex(pattern))) return m.str();
    return "";
}

// All occurrences of pattern in s.
// Note: sregex_iterator requires a named regex; a temporary is a compile error in C++20+.
vector<string> findAll(string s, string pattern) {
    vector<string> results;
    regex re(pattern);
    sregex_iterator it(s.begin(), s.end(), re);
    sregex_iterator end;
    while (it != end) {
        results.push_back(it->str());
        it++;
    }
    return results;
}

/*
Common patterns:

Integer validation:
  regex_match(s, regex(R"(^-?\d+$)"))            // integer (possibly negative)
  regex_match(s, regex(R"(^\d+$)"))              // non-negative integer
  regex_match(s, regex(R"(^[1-9]\d*$)"))         // integer without leading zeros

Decimal number:
  regex_match(s, regex(R"(^-?\d+(\.\d+)?$)"))    // integer or decimal
  regex_match(s, regex(R"(^-?\d+\.\d+$)"))       // strictly decimal (has dot)

Subscript/superscript (e.g. Bad Latex):
  regex_match(s, regex(R"([a-zA-Z0-9]+_\{\d+\})"))
       // X_{Y} where Y is integer
  regex_match(s, regex(R"([a-zA-Z0-9]+\^\{\d+\})"))
       // X^{Y} where Y is integer
  findAll(s, R"([a-zA-Z0-9]+[_^]\{\d+\})")
       // all sub/superscripts

Extract numbers:
  findAll(s, R"(\d+)")                           // sequences of digits
  findAll(s, R"(-?\d+(\.\d+)?)")                 // integers and decimals

```

```
Other:
  regex_match(s, regex(R"(^[a-zA-Z]+$)"))       //
   only letters
  regex_match(s, regex(R"(^[a-zA-Z0-9]+$)"))    //
   alphanumeric
  regex_match(s, regex(R"(^[a-zA-Z0-9 ]+$)"))   //
   alphanumeric with spaces
*/
```

Listing 5: Patterns for validating integers, decimals, and subscript/superscript formats, plus the helpers `findFirst` and `findAll`. Complexity: $O(n)$ per search for simple patterns (n = length of the string); worst case exponential due to the backtracking engine. Prefer compiling the regex once ($O(p)$, p = pattern length) and reusing it. Requires C++11 or later.